Text Mining - Final Project: Airbnb Price Prediction - London

Alvaro Camacho, David Permutti, Sanghmitra, Kant Tanyawong, Pol Tawin Choksitthikasam

2026-02-25

Contents

SECTION 0 — Load, Explore, and Prepare Airbnb NYC 2019 Data **2**
0.1 Load Libraries and Functions . . . 2
0.2. Load Data + Column Names . . . 2
0.3. Clean Price Column + Filter for Valid Rows . . . 4
0.4. Price Variable Exploration . . . 4
0.5. "name" Variable Exploration . . . 9
0.6. "description" Variable Exploration . . . 10

SECTION 1 - Train/Test Split **11**
1.1. Create Master Modeling Dataset . . . 11
1.2. Train/Test Split (Stratified on price_log) . . . 11
1.3. Verify Distribution Similarity . . . 12

SECTION 2 - "name" Baseline Text Model **13**
2.1. Define "name" n-Gram Model Target Variables . . . 13
2.2. Builde 1:2 n-Gram Model . . . 13
2.3. Train LASSO Model . . . 14
2.4. Model Evaluation Plots . . . 14
2.5. Baseline Test Set Performance . . . 16

SECTION 3 - n-Gram Model Exploration on Descriptions **16**
3.1. Helper Function . . . 16
3.2. Run n-Gram Models for 1, 1:2, 1:3 . . . 17
3.3. Plot Model Performances . . . 17
3.4. Choose Best n-Gram and . . . 18

SECTION 4 - Embeddings Model **20**
4.1. Project Descriptions into Embedding Space . . . 20
4.2. Fit LASSO Embeddings Model . . . 21
4.3. Fit LASSO Embeddings + 1:3 N-Grams . . . 22
4.4. Compare Models (Embeddings, Embeddings+1:3 Grams, 1:3 Grams (baseline)) . . . 23
4.5. Diagnostics Plots for Best Model . . . 25

SECTION 5 - Structured Features + Combined Model **27**
5.1. Structured-Only Model . . . 27
5.2. Structured + Tex Model . . . 29
5.3. Model Comparison . . . 31
5.4. . . . 31

SECTION 0 — Load, Explore, and Prepare Airbnb NYC 2019 Data

0.1 Load Libraries and Functions

```
library(tidyverse)
library(quanteda)
library(textclean)
library(glmnet)
library(ggrepel)
library(stringi)
library(rsample)   # train/test split


source("TMEF_dfm.R")
source("vectorFunctions.R")

vecSmall = readRDS("data/vecSmall.RDS")
if (!inherits(vecSmall, "data.table")) vecSmall = as.data.table(vecSmall)

load("data/wfFile.RData")
```

0.2. Load Data + Column Names

```
london_raw = data.table::fread("london.csv.gz", encoding = "UTF-8")
london_raw = as_tibble(london_raw)

colnames(london_raw)
```

```
##  [1] "id"
##  [2] "listing_url"
##  [3] "scrape_id"
##  [4] "last_scraped"
##  [5] "source"
##  [6] "name"
##  [7] "description"
##  [8] "neighborhood_overview"
##  [9] "picture_url"
## [10] "host_id"
## [11] "host_url"
## [12] "host_name"
## [13] "host_since"
## [14] "host_location"
## [15] "host_about"
## [16] "host_response_time"
## [17] "host_response_rate"
## [18] "host_acceptance_rate"
## [19] "host_is_superhost"
## [20] "host_thumbnail_url"
## [21] "host_picture_url"
## [22] "host_neighbourhood"
## [23] "host_listings_count"
## [24] "host_total_listings_count"
## [25] "host_verifications"
## [26] "host_has_profile_pic"
```

```
## [27] "host_identity_verified"
## [28] "neighbourhood"
## [29] "neighbourhood_cleansed"
## [30] "neighbourhood_group_cleansed"
## [31] "latitude"
## [32] "longitude"
## [33] "property_type"
## [34] "room_type"
## [35] "accommodates"
## [36] "bathrooms"
## [37] "bathrooms_text"
## [38] "bedrooms"
## [39] "beds"
## [40] "amenities"
## [41] "price"
## [42] "minimum_nights"
## [43] "maximum_nights"
## [44] "minimum_minimum_nights"
## [45] "maximum_minimum_nights"
## [46] "minimum_maximum_nights"
## [47] "maximum_maximum_nights"
## [48] "minimum_nights_avg_ntm"
## [49] "maximum_nights_avg_ntm"
## [50] "calendar_updated"
## [51] "has_availability"
## [52] "availability_30"
## [53] "availability_60"
## [54] "availability_90"
## [55] "availability_365"
## [56] "calendar_last_scraped"
## [57] "number_of_reviews"
## [58] "number_of_reviews_ltm"
## [59] "number_of_reviews_l30d"
## [60] "availability_eoy"
## [61] "number_of_reviews_ly"
## [62] "estimated_occupancy_l365d"
## [63] "estimated_revenue_l365d"
## [64] "first_review"
## [65] "last_review"
## [66] "review_scores_rating"
## [67] "review_scores_accuracy"
## [68] "review_scores_cleanliness"
## [69] "review_scores_checkin"
## [70] "review_scores_communication"
## [71] "review_scores_location"
## [72] "review_scores_value"
## [73] "license"
## [74] "instant_bookable"
## [75] "calculated_host_listings_count"
## [76] "calculated_host_listings_count_entire_homes"
## [77] "calculated_host_listings_count_private_rooms"
## [78] "calculated_host_listings_count_shared_rooms"
## [79] "reviews_per_month"
```

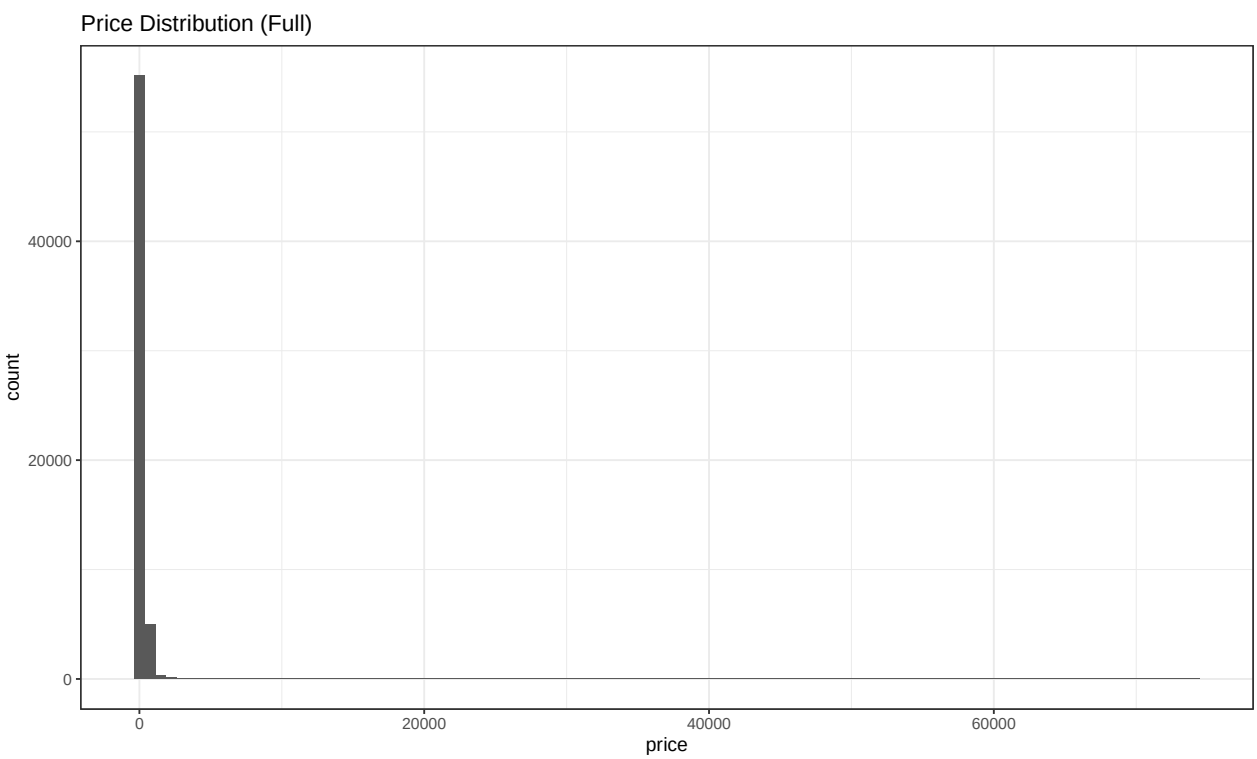
0.3. Clean Price Column + Filter for Valid Rows

Restrictions: - price must be numeric and > 0 - name must be non-missing and non-empty - description must be non-missing and non-empty

```
price_num = suppressWarnings(
  as.numeric(gsub("[^0-9.]", "", london_raw$price))
)

london_clean = london_raw %>%
  mutate(
    name = as.character(name),
    description = as.character(description),
    price = price_num
  ) %>%
  filter(
    !is.na(price), price > 0,
    !is.na(name), name != "",
    !is.na(description), description != ""
  )
```

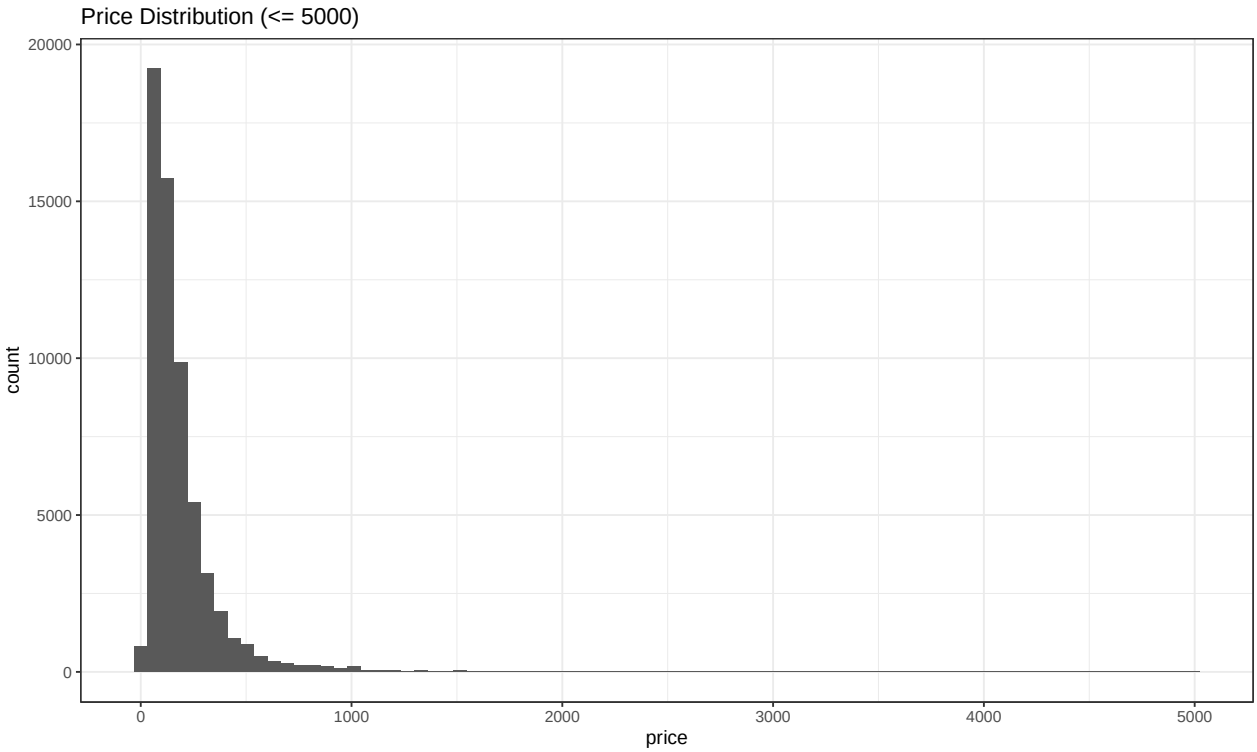
0.4. Price Variable Exploration

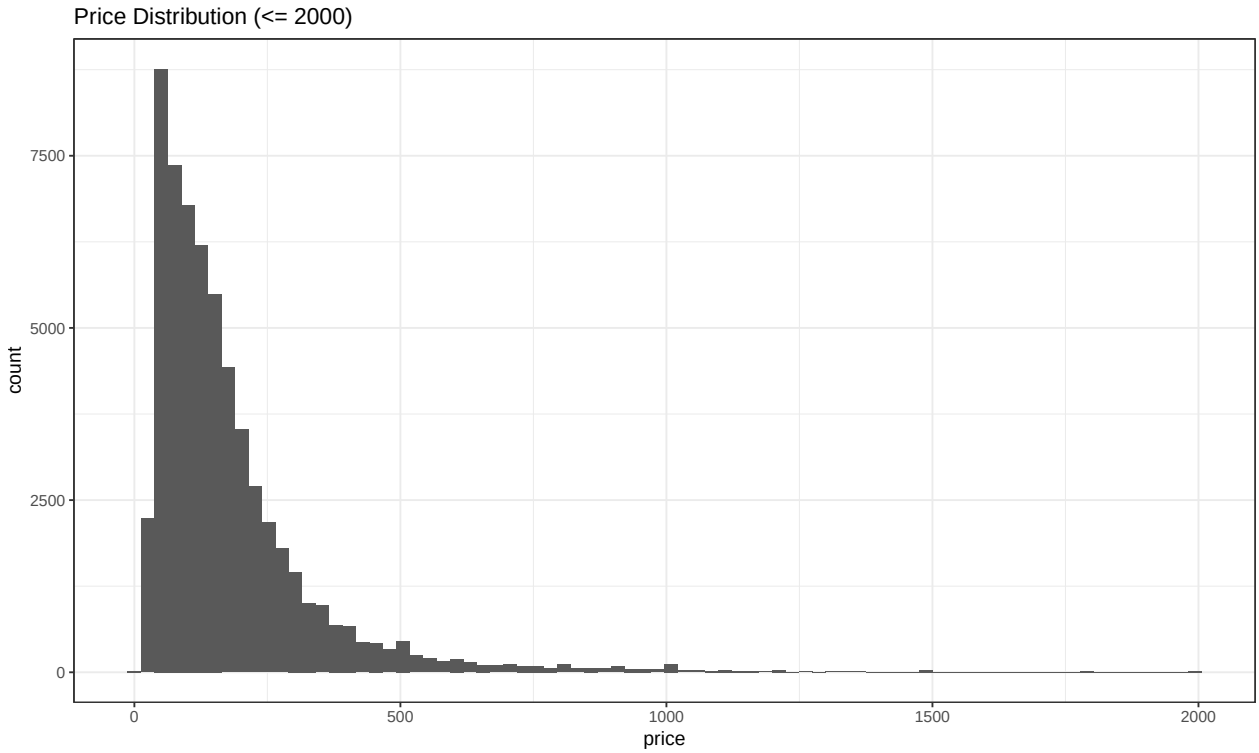
```
ggplot(london_clean, aes(x = price)) +
  geom_histogram(bins = 100) +
  theme_bw() +
  labs(title = "Price Distribution (Full)", x = "price", y = "count")
```

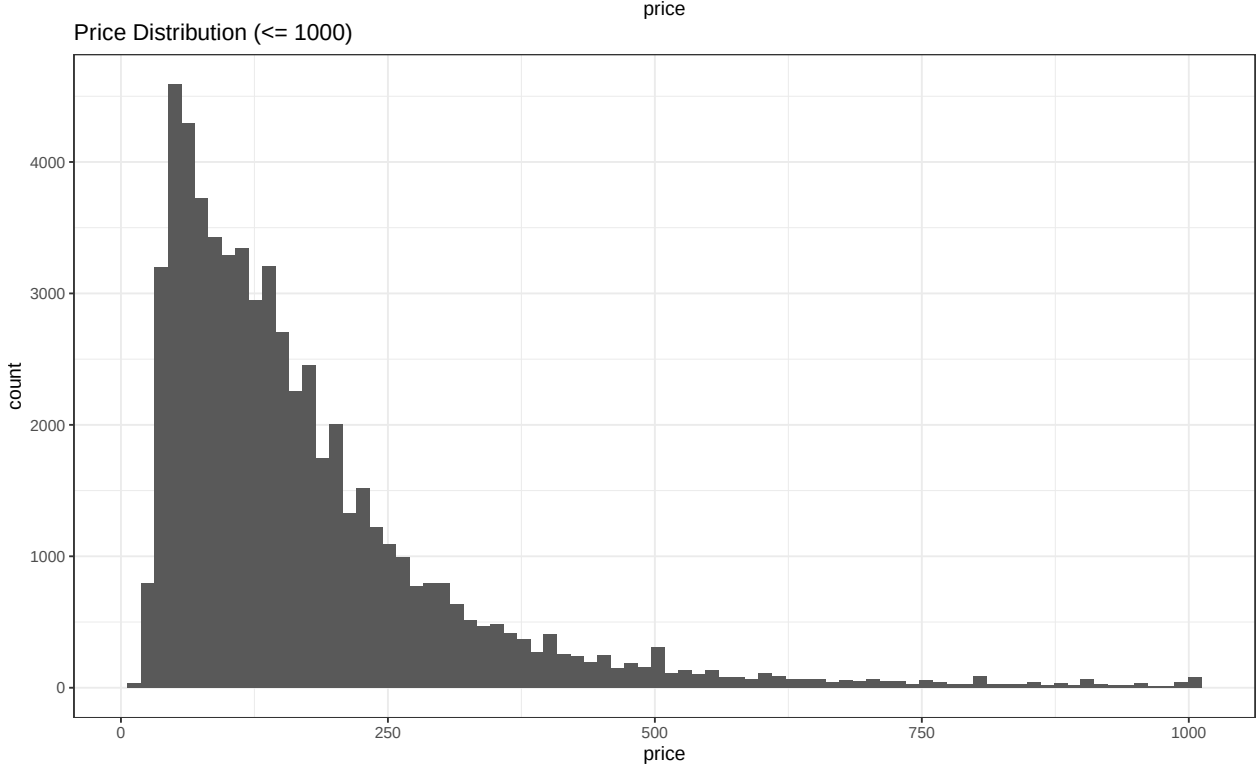


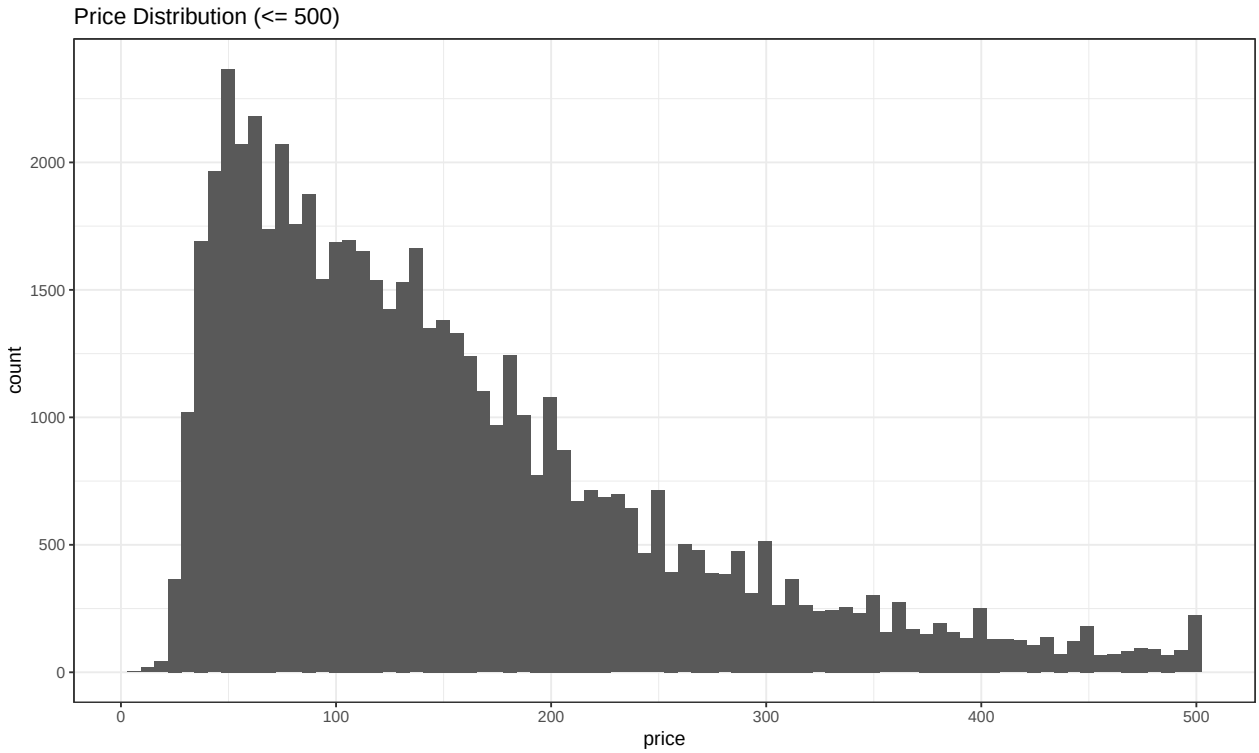

```
ranges = c(5000, 2000, 1000, 500, 300, 200)

for (r in ranges) {
```

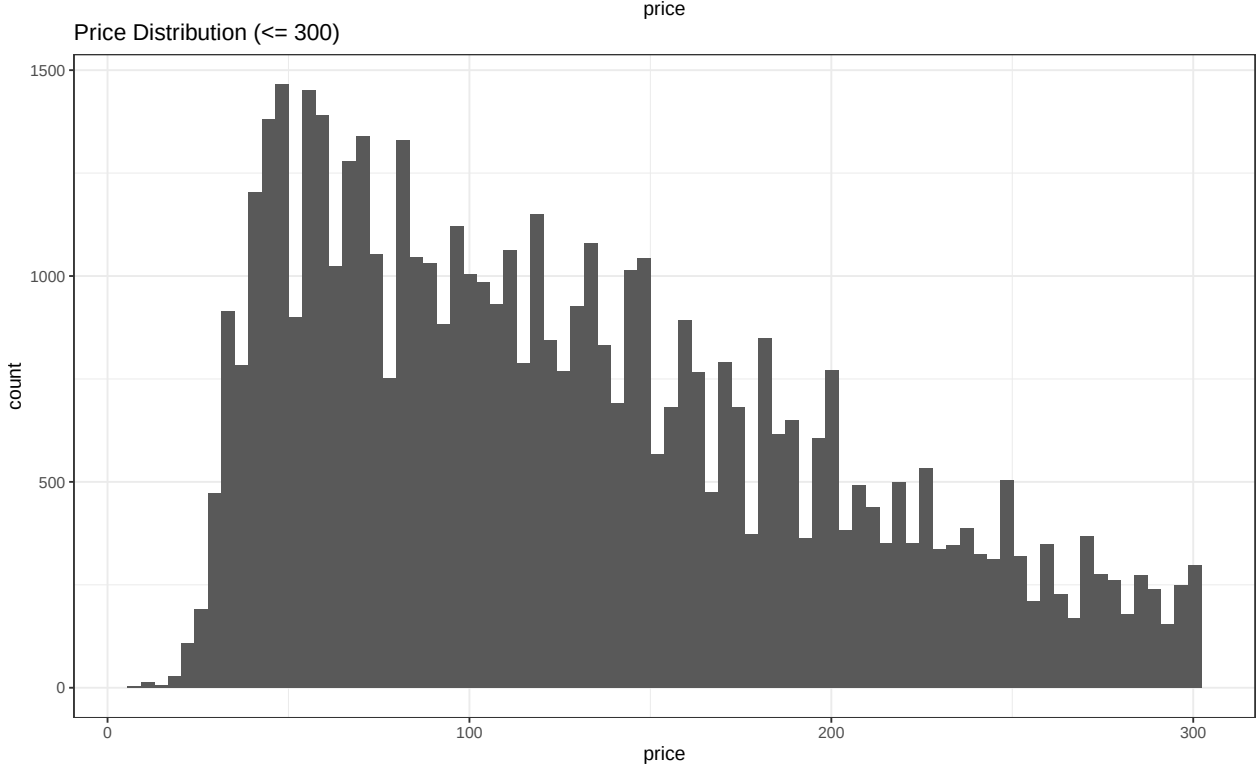
```r
  print(
    ggplot(london_clean %>% filter(price <= r), aes(x = price)) +
      geom_histogram(bins = 80) +
      theme_bw() +
      labs(
        title = paste("Price Distribution (<= ", r, ")", sep = ""),
        x = "price",
        y = "count"
      )
  )
}
```

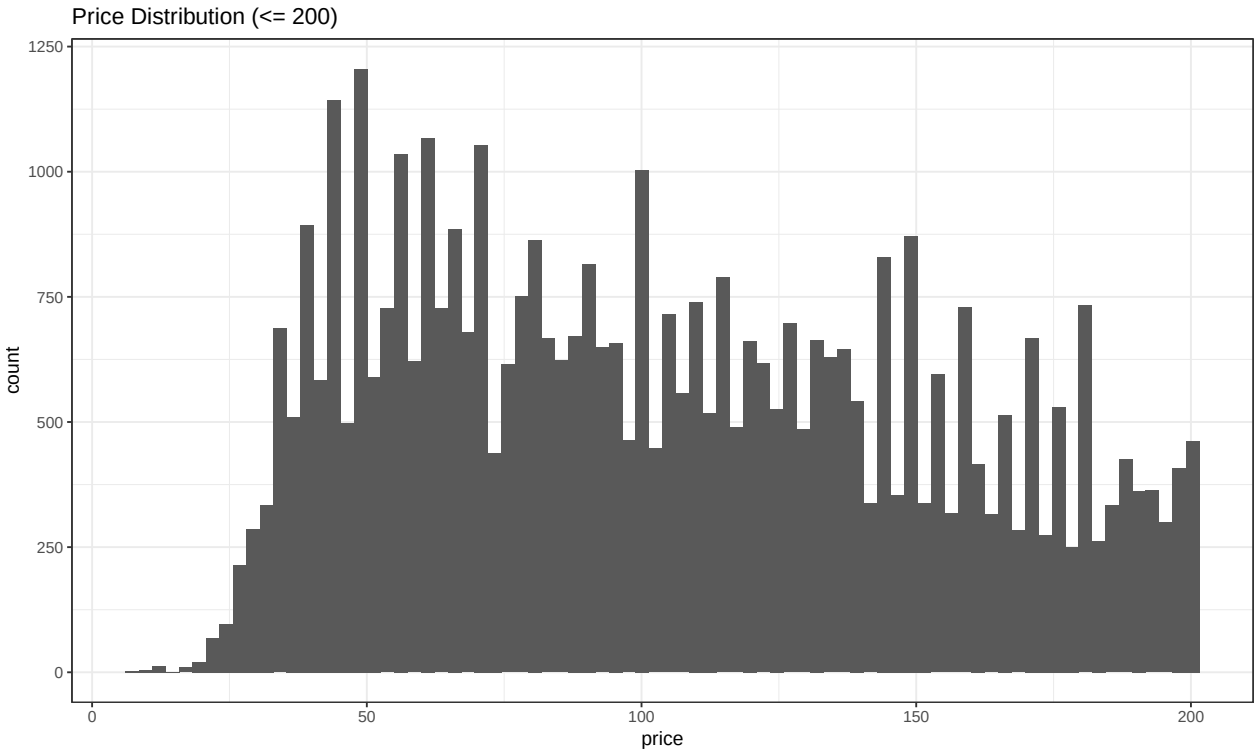



Price Distribution (<= 2000)
count
7500
5000
2500
0
0
500
1000
1500
2000
price

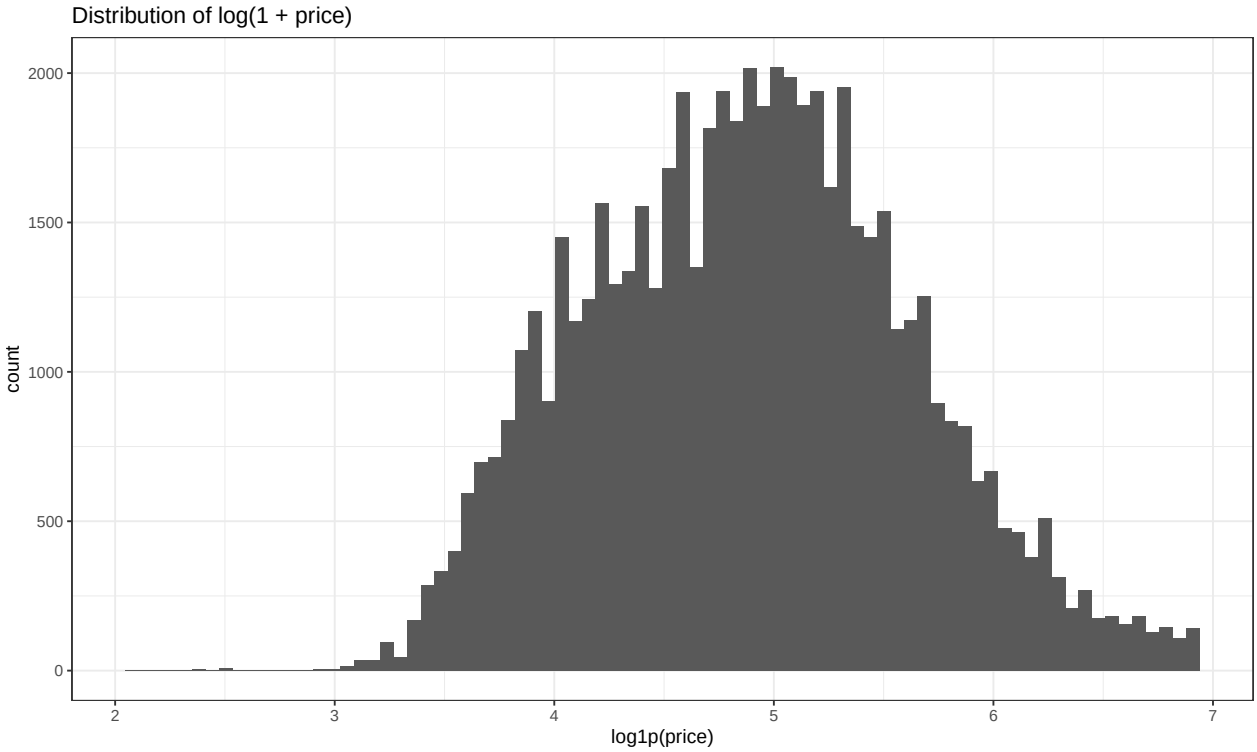

Price Distribution (<= 1000)
count
4000
3000
2000
1000
0
0
250
500
750
1000
price


Price Distribution (<= 500)
count
price
0
500
1000
1500
2000
0
100
200
300
400
500


Price Distribution (<= 300)
count
price
0
500
1000
1500
0
100
200
300




```
ggplot(london_clean%>% filter(price <= 1000), aes(x = log1p(price))) +
  geom_histogram(bins = 80) +
  theme_bw() +
  labs(title = "Distribution of log(1 + price)",
       x = "log1p(price)")
```



We restrict analysis to listings priced below $1000 to remove ultra-luxury outliers that represent a distinct pricing regime and may unduly influence model estimation. The log transformation is then applied to stabilize variance.

0.5. "name" Variable Exploration

We now explore the textual richness of the "name" field. The purpose of this section is to assess whether listing titles alone provide sufficient linguistic depth for predictive modeling.

```
has_non_ascii_name = stringi::stri_detect_regex(london_clean$name, "[^\\x00-\\x7F]")

mean(has_non_ascii_name)
```

0.5.1. ASCII vs. non-ASCII Characters

```
## [1] 0.02594277
```

```
sum(has_non_ascii_name)
```

```
## [1] 1574
```

We compute the proportion of listings containing non-ASCII characters. This helps determine whether special characters, accents, or encoding issues are present that may affect tokenization.

```
ascii_count_name = stringi::stri_count_regex(london_clean$name, "[\\x00-\\x7F]")
total_chars_name = nchar(london_clean$name)

ascii_share_name = ifelse(total_chars_name > 0,
                          ascii_count_name / total_chars_name,
                          NA_real_)

summary(ascii_share_name)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.0000  1.0000  1.0000  0.9987  1.0000  1.0000
```

This measures the share of ASCII characters per listing title. If most titles are nearly 100% ASCII, preprocessing complexity is minimal.

0.5.2. Word Count in "name"

We compute the number of words per listing title.

This is crucial because:

- Very short documents (e.g., 3–5 words)
- Provide limited linguistic variability
- Restrict n-gram richness
- Reduce the expressive power of embeddings

```
name_word_count = stringi::stri_count_words(london_clean$name)

summary(name_word_count)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.000   5.000   6.000   6.034   7.000  45.000
```

```
mean(name_word_count)
```

```
## [1] 6.033607
```

0.5.3. Conclusions of "name" Variable The listing title field:

- Is almost entirely ASCII (clean corpus)
- Contains very short documents (mean 6 words)
- Provides limited linguistic depth

This means that name is unlikely to constitute a sufficiently rich standalone corpus for demonstrating advanced text mining tools

0.6. "description" Variable Exploration

We now explore the textual richness of the "description" field. The purpose of this section is to assess whether listing descriptions alone provide sufficient linguistic depth for predictive modeling.

```
has_non_ascii_desc = stringi::stri_detect_regex(london_clean$description, "[^\\x00-\\x7F]")

mean(has_non_ascii_desc)
```

0.6.1. ASCII vs. non-ASCII Characters

```
## [1] 0.2822389
```

```
sum(has_non_ascii_desc)
```

```
## [1] 17124
```

We compute the proportion of listings containing non-ASCII characters. This helps determine whether special characters, accents, or encoding issues are present that may affect tokenization.

```
ascii_count_desc = stringi::stri_count_regex(london_clean$description, "[\\x00-\\x7F]")
total_chars_desc = nchar(london_clean$description)

ascii_share_desc = ifelse(total_chars_desc > 0,
                          ascii_count_desc / total_chars_desc,
                          NA_real_)

summary(ascii_share_desc)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6446  0.9980  1.0000  0.9980  1.0000  1.0000
```

This measures the share of ASCII characters per listing title. If most titles are nearly 100% ASCII, preprocessing complexity is minimal.

0.6.2. Word Count in "description" We compute the number of words per listing title.

This is crucial because:

- Very short documents (e.g., 3–5 words)
- Provide limited linguistic variability
- Restrict n-gram richness
- Reduce the expressive power of embeddings

```
desc_word_count = stringi::stri_count_words(london_clean$description)

summary(desc_word_count)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    0.00   48.00   73.00   63.76   83.00  196.00
```

```r
mean(desc_word_count)
```

```
## [1] 63.75722
```

0.6.3. Conclusions of "description" Variable The listing description field:

- Is almost entirely ASCII (clean corpus)
- Contains substantially longer documents (mean title length)
- Provides significantly greater linguistic depth
- Exhibits high vocabulary diversity across 50,000+ listings

Forms a large corpus suitable for advanced text mining techniques

This means that description constitutes a sufficiently rich standalone corpus for demonstrating advanced text mining tools such as n-grams, embeddings, and regularized regression. Unlike listing titles, descriptions provide the semantic depth necessary to extract meaningful textual patterns related to pricing.

SECTION 1 - Train/Test Split

1.1. Create Master Modeling Dataset

```r
# 1.1 Create Master Modeling Dataset
# We keep the same restrictions as Section 0 (already in london_clean):
# - price > 0
# - name non-empty
# - description non-empty
# We also apply the same outlier rule as NYC (price < 1000), then log-transform.

london_master = london_clean %>%
  filter(price < 1000) %>%
  mutate(price_log = log1p(price))

summary(london_master$price)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##       7      76     133     172     216     999
```

```r
summary(london_master$price_log)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   2.079   4.344   4.898   4.885   5.380   6.908
```

1.2. Train/Test Split (Stratified on price_log)

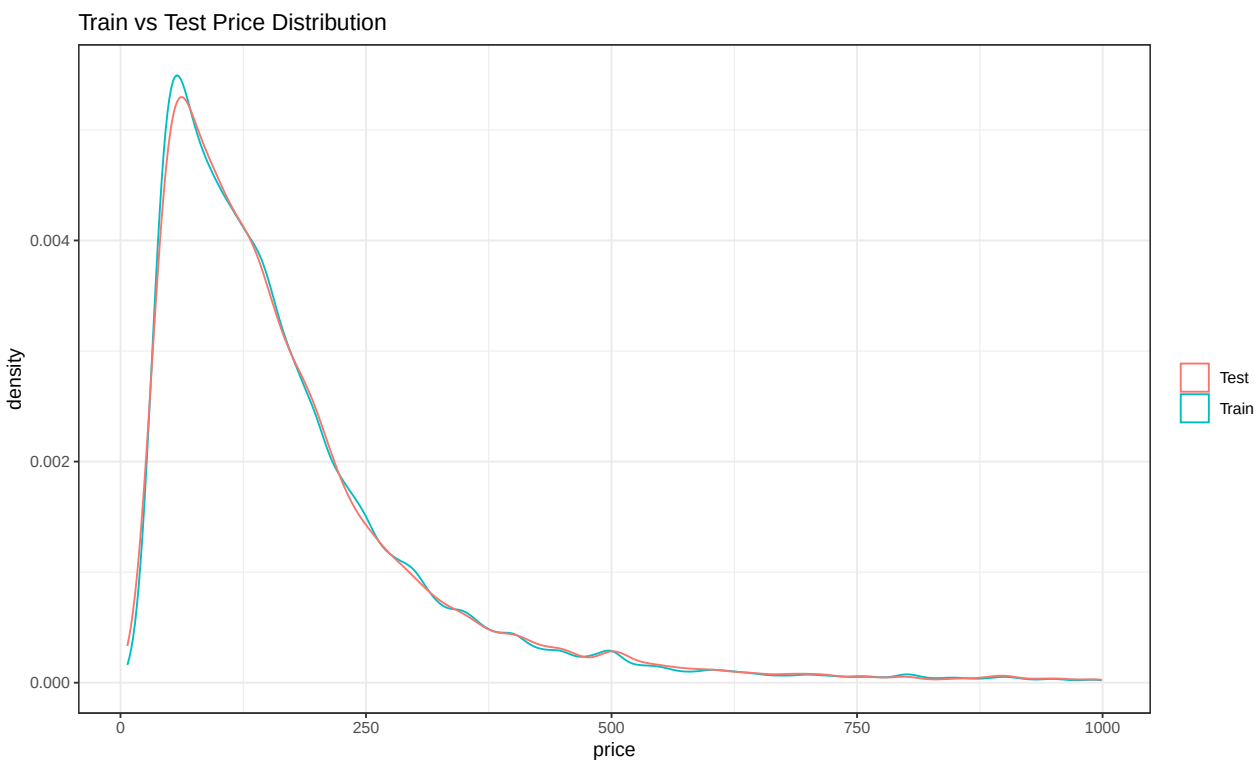
```r
# 1.2 Train/Test Split (stratified on price_log)

set.seed(02138)

split = initial_split(
  london_master,
  prop = 0.8,
  strata = price_log
)

london_train = training(split)
```

```r
london_test = testing(split)

c(n_train = nrow(london_train), n_test = nrow(london_test))
```

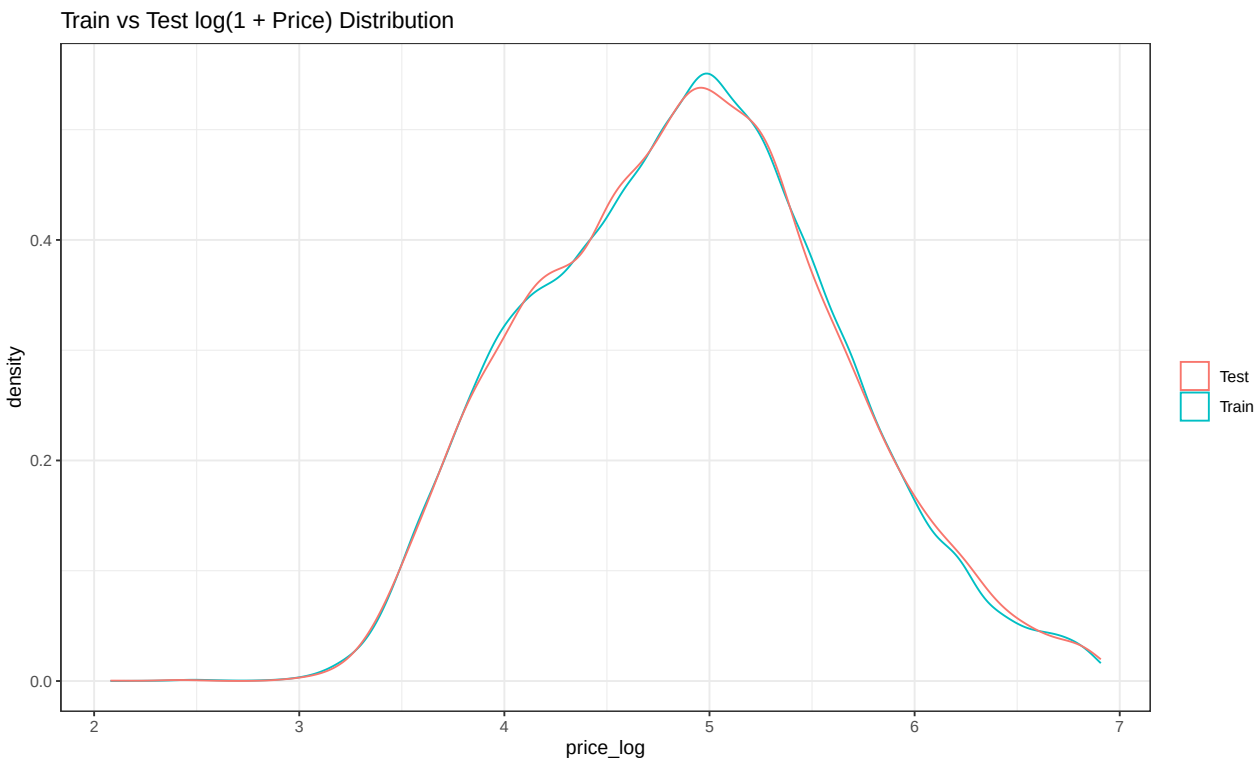
```
## n_train  n_test
##   47925   11983
```

1.3. Verify Distribution Similarity

```r
# 1.3 Verify Distribution Similarity

ggplot() +
  geom_density(data = london_train, aes(x = price, color = "Train")) +
  geom_density(data = london_test, aes(x = price, color = "Test")) +
  theme_bw() +
  labs(title = "Train vs Test Price Distribution", x = "price", color = "")
```




```r
ggplot() +
  geom_density(data = london_train, aes(x = price_log, color = "Train")) +
  geom_density(data = london_test, aes(x = price_log, color = "Test")) +
  theme_bw() +
  labs(title = "Train vs Test log(1 + Price) Distribution", x = "price_log", color = "")
```



SECTION 2 - "name" Baseline Text Model

2.1. Define "name" n-Gram Model Target Variables

```
train_text_name = london_train$name
test_text_name = london_test$name

train_Y = london_train$price_log
test_Y = london_test$price_log
```

2.2. Builde 1:2 n-Gram Model

```
best_ngram = 1:2

dfm_train_name = TMEF_dfm(train_text_name, ngrams = best_ngram)

dfm_test_name = TMEF_dfm(
  test_text_name,
  ngrams = best_ngram,
  min.prop = 0
) %>%
  dfm_match(colnames(dfm_train_name))

X_train_name = as.matrix(dfm_train_name)
X_test_name = as.matrix(dfm_test_name)

dim(X_train_name)
```

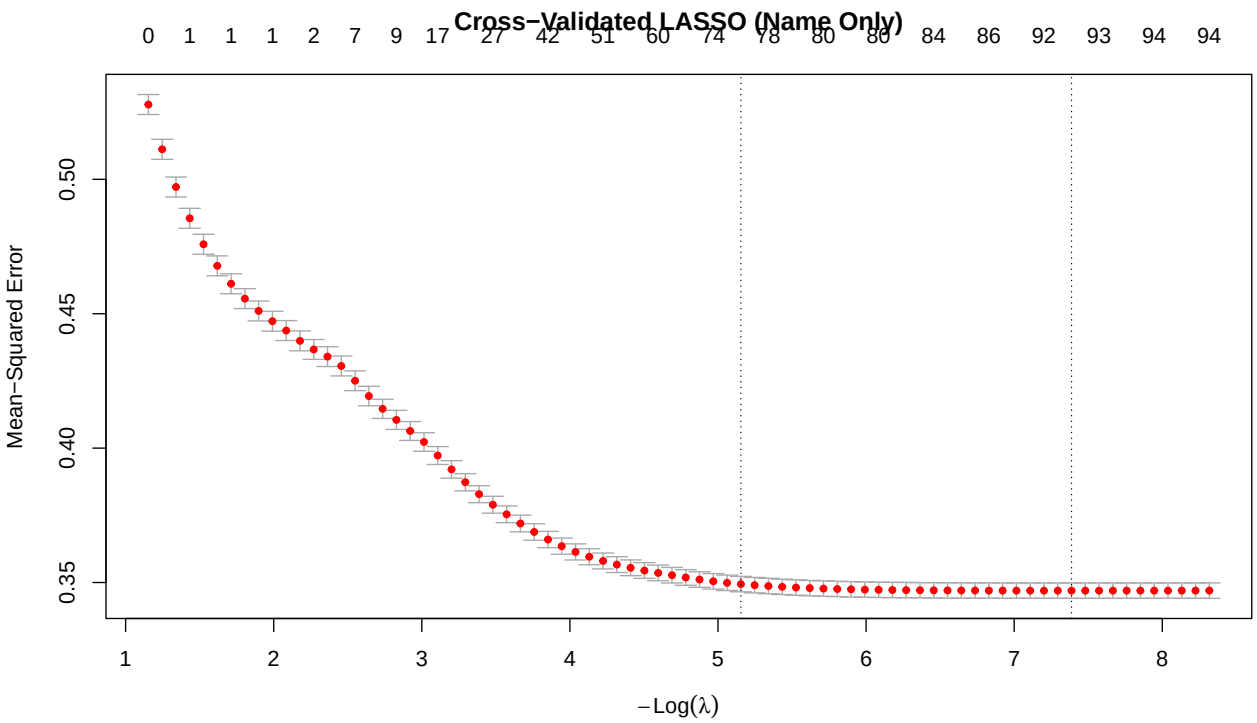
```
## [1] 47925   100
```

2.3. Train LASSO Model

```
lasso_name = cv.glmnet(
  x = X_train_name,
  y = train_Y
)
```

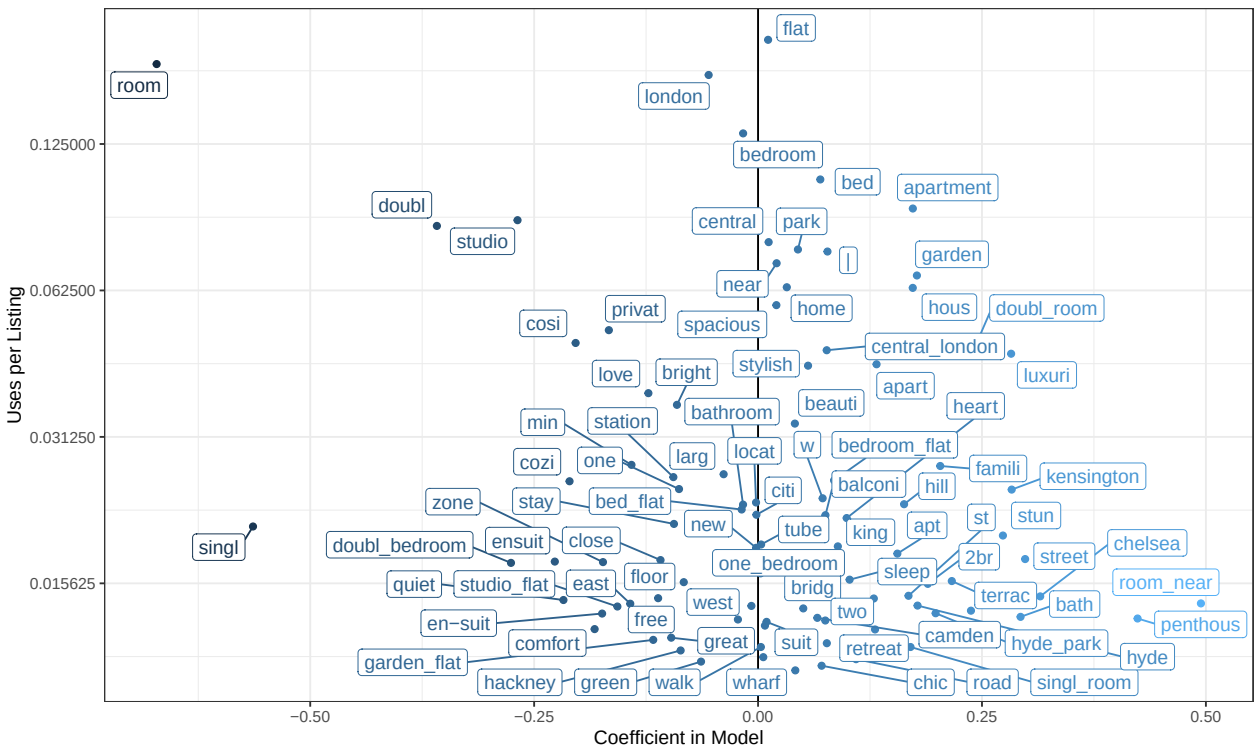
2.4. Model Evaluation Plots

```
plot(lasso_name)
title("Cross-Validated LASSO (Name Only)")
```




```
plotCoefs = lasso_name %>%
  coef(s = "lambda.min") %>%
  drop() %>%
  as.data.frame() %>%
  rownames_to_column("ngram") %>%
  rename(score = ".") %>%
  filter(score != 0,
         ngram != "(Intercept)",
         !is.na(score))

# Compute average frequency of each term
plotFreqs = data.frame(
  ngram = colnames(X_train_name),
  freq  = colMeans(X_train_name)
)
```

```r
# Merge
plotDat = plotCoefs %>%
  left_join(plotFreqs, by = "ngram")

# Plot
plotDat %>%
  ggplot(aes(x = score,
             y = freq,
             label = ngram,
             color = score)) +
  geom_vline(xintercept = 0) +
  geom_point() +
  geom_label_repel(max.overlaps = 15) +
  scale_y_continuous(trans = "log2") +
  theme_bw() +
  labs(x = "Coefficient in Model",
       y = "Uses per Listing") +
  theme(legend.position = "none")
```



The cross-validated LASSO curve shows that listing titles contain some predictive signal. As the regularization penalty decreases, model error declines steadily, indicating that certain title words help explain variation in prices. The model selects a moderate number of features (approximately 70–90), suggesting that not all title words are informative, but some do contribute meaningfully.

The coefficient plot further confirms this. A small number of economically intuitive words—such as property type and location indicators—receive larger coefficients, while the majority of title terms cluster near zero. This indicates that predictive power is concentrated in a limited set of keywords rather than distributed broadly across the vocabulary.

Overall, listing titles encode structured economic information (e.g., location and property characteristics), but their short average length (approximately six words) limits linguistic richness. As a result, while titles

provide some predictive value, they do not constitute a sufficiently rich standalone corpus for demonstrating more advanced text mining techniques.

2.5. Baseline Test Set Performance

```
rmse = function(y, yhat) sqrt(mean((yhat - y)^2))
mae = function(y, yhat) mean(abs(yhat - y))

pred_log_name = predict(lasso_name, newx = X_test_name)[,1]

pred_price_name = expm1(pred_log_name)
actual_price = expm1(test_Y)

metrics_name = tibble(
  model = "Name Only (1:2 n-grams)",
  rmse_log = rmse(test_Y, pred_log_name),
  mae_log = mae(test_Y, pred_log_name),
  rmse_dollar = rmse(actual_price, pred_price_name),
  mae_dollar = mae(actual_price, pred_price_name)
)

metrics_name
```

```
## # A tibble: 1 x 5
##   model                    rmse_log mae_log rmse_dollar mae_dollar
##   <chr>                       <dbl>   <dbl>       <dbl>      <dbl>
## 1 Name Only (1:2 n-grams)    0.591   0.454        133.       77.6
```

The name-only model achieves measurable predictive performance on the held-out test set (RMSE_log 0.59; RMSE_$ 133). This confirms that listing titles contain structured economic information, particularly related to property type and location. However, the magnitude of prediction error remains substantial, indicating that titles alone do not capture enough variation in listing prices. Given their short average length (~6 words), titles provide limited linguistic depth and therefore constitute a weak standalone corpus for advanced text mining analysis.

SECTION 3 - n-Gram Model Exploration on Descriptions

3.1. Helper Function

```
rmse = function(y, yhat) sqrt(mean((yhat - y)^2))
mae = function(y, yhat) mean(abs(yhat - y))

evaluate_desc_ngram = function(ngr_value){

  # Build dfm on TRAIN only (keeps trim based on train)
  dfm_train = TMEF_dfm(london_train$description, ngrams = ngr_value)

  # Build dfm on TEST, then match to train features
  dfm_test = TMEF_dfm(london_test$description,
                      ngrams = ngr_value,
                      min.prop = 0) %>%
    dfm_match(colnames(dfm_train))

  trainX = dfm_train %>% as.matrix()
```

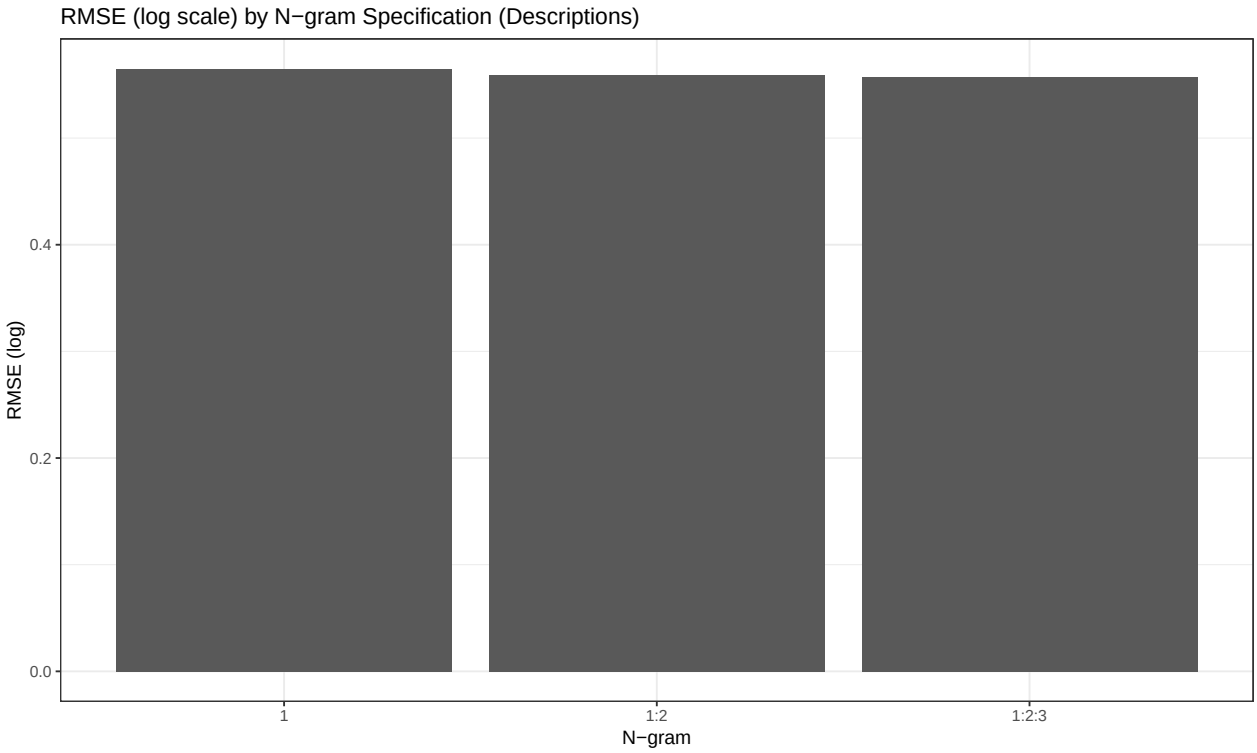
```r
  testX = dfm_test %>% as.matrix()

  trainY = london_train$price_log
  testY = london_test$price_log

  # LASSO
  lasso_model = cv.glmnet(x = trainX, y = trainY)

  # Predict on test
  pred_log = predict(lasso_model, newx = testX)[,1]

  # Back-transform to dollars
  pred_price = expm1(pred_log)
  actual_price = expm1(testY)

  # Return metrics + objects (for plots if needed)
  list(
    metrics = tibble(
      ngram = paste(ngr_value, collapse = ":"),
      rmse_log = rmse(testY, pred_log),
      mae_log = mae(testY, pred_log),
      rmse_dollar = rmse(actual_price, pred_price),
      mae_dollar = mae(actual_price, pred_price),
      n_features = ncol(trainX)
    ),
    model = lasso_model,
    trainX = trainX
  )
}
```

3.2. Run n-Gram Models for 1, 1:2, 1:3

```r
res_desc_1 = evaluate_desc_ngram(1)
res_desc_12 = evaluate_desc_ngram(1:2)
res_desc_123 = evaluate_desc_ngram(1:3)

results_desc = bind_rows(
  res_desc_1$metrics,
  res_desc_12$metrics,
  res_desc_123$metrics
)
```

3.3. Plot Model Performances

```r
results_desc %>%
  ggplot(aes(x = ngram, y = rmse_log)) +
  geom_col() +
  theme_bw() +
  labs(title = "RMSE (log scale) by N-gram Specification (Descriptions)",
       x = "N-gram",
       y = "RMSE (log)")
```

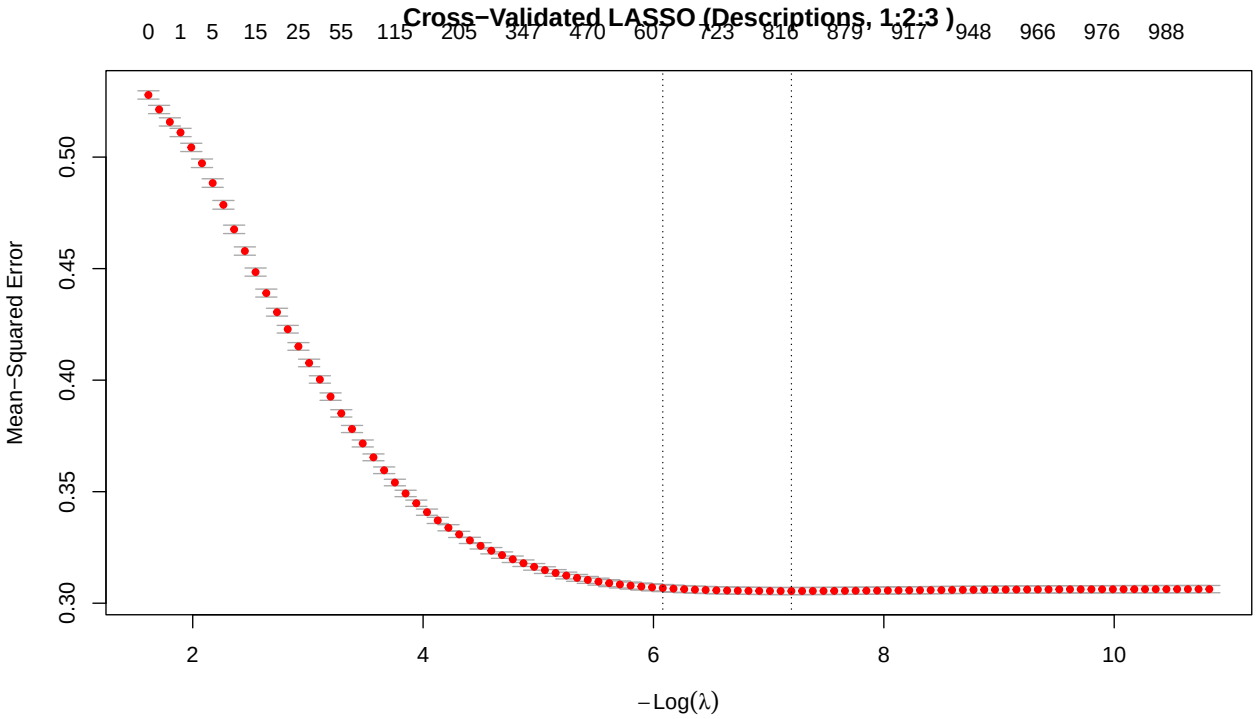


3.4. Choose Best n-Gram and

```
best_row = results_desc %>% arrange(rmse_log) %>% slice(1)
best_row
```

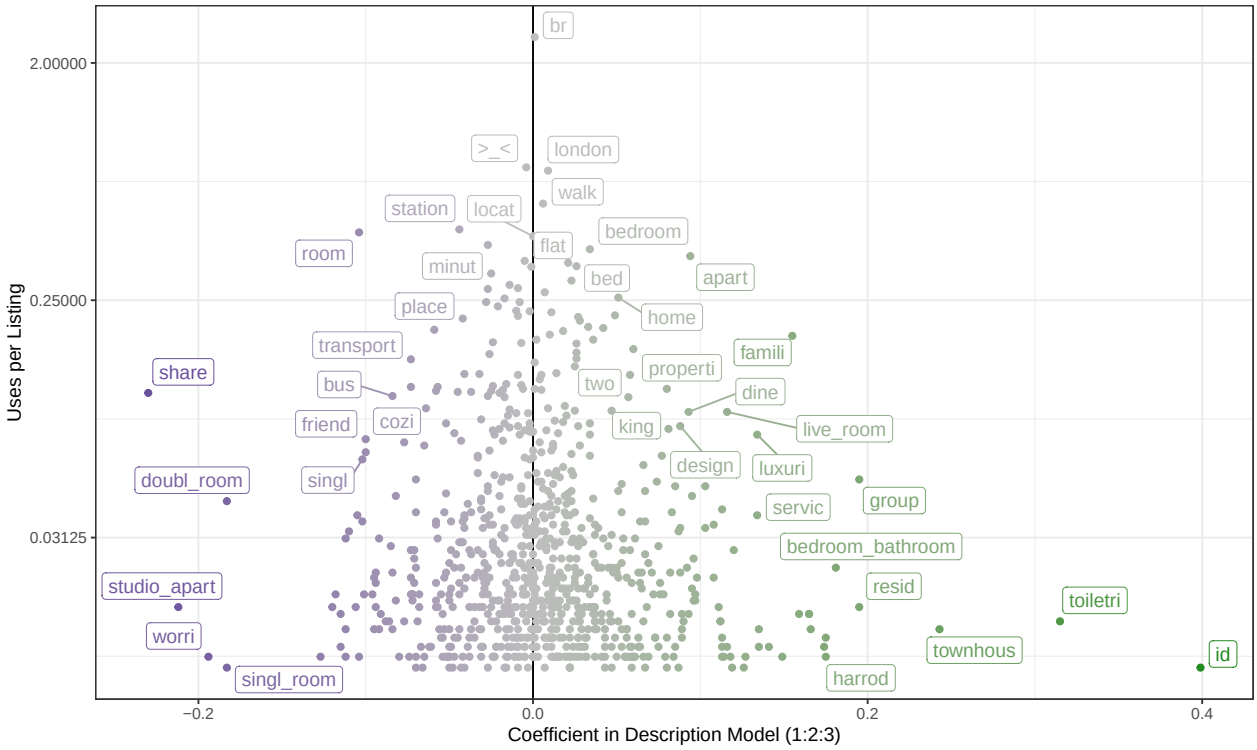
```
## # A tibble: 1 x 6
##   ngram rmse_log mae_log rmse_dollar mae_dollar n_features
##   <chr>    <dbl>   <dbl>       <dbl>      <dbl>      <int>
## 1 1:2:3    0.557   0.431        124.       73.3       1012
```

```
best_ngram = best_row$ngram

# Convert back into numeric vector
best_ngram_value = as.numeric(strsplit(best_ngram, ":")[[1]])

# Get the stored result object
best_res = if(best_ngram == "1") res_desc_1 else if(best_ngram == "1:2") res_desc_12 else res_desc_123

plot(best_res$model)
title(paste("Cross-Validated LASSO (Descriptions,", best_ngram, ")", sep = " "))
```




```
plotCoefs = best_res$model %>%
  coef(s = "lambda.min") %>%
  drop() %>%
  as.data.frame() %>%
  rownames_to_column(var = "ngram") %>%
  rename(score = ".") %>%
  filter(score != 0 & ngram != "(Intercept)" & !is.na(score))

plotDat = plotCoefs %>%
  left_join(
    data.frame(
      ngram = colnames(best_res$trainX),
      freq = colMeans(best_res$trainX)
    ),
    by = "ngram"
  ) %>%
  mutate_at(vars(score, freq), ~round(., 3))

plotDat %>%
  ggplot(aes(x = score, y = freq, label = ngram, color = score)) +
  scale_color_gradient2(low = "navyblue",
                        mid = "grey",
                        high = "forestgreen",
                        midpoint = 0) +
  geom_vline(xintercept = 0) +
  geom_point() +
  geom_label_repel(max.overlaps = 15) +
  scale_y_continuous(trans = "log2") +
  theme_bw() +
  labs(x = paste("Coefficient in Description Model (", best_row$ngram, ")", sep=""),
       y = "Uses per Listing") +
```

```
  theme(legend.position = "none")
```



The coefficient–frequency plot for the 1:2:3 description model reveals economically interpretable signals. High-end property descriptors and premium location indicators receive positive coefficients, while terms associated with shared or smaller accommodations receive negative coefficients. Although most phrases cluster near zero, a meaningful subset of n-grams contributes to price prediction. This confirms that description text contains structured economic information, though its predictive contribution remains limited relative to overall price variance.

SECTION 4 - Embeddings Model

4.1. Project Descriptions into Embedding Space

```
all_text = c(london_train$description, london_test$description)

vdat = vecCheck(
  all_text,
  vecSmall,
  wfFile,
  PCAtrim = 1
)
```

```
## ==============================================================================
```

```
n_train = nrow(london_train)

vdat_train = vdat[1:n_train, ]
vdat_test  = vdat[(n_train + 1):nrow(vdat), ]

dim(vdat_train)
```

```
## [1] 47925   300
```

```r
dim(vdat_test)
```

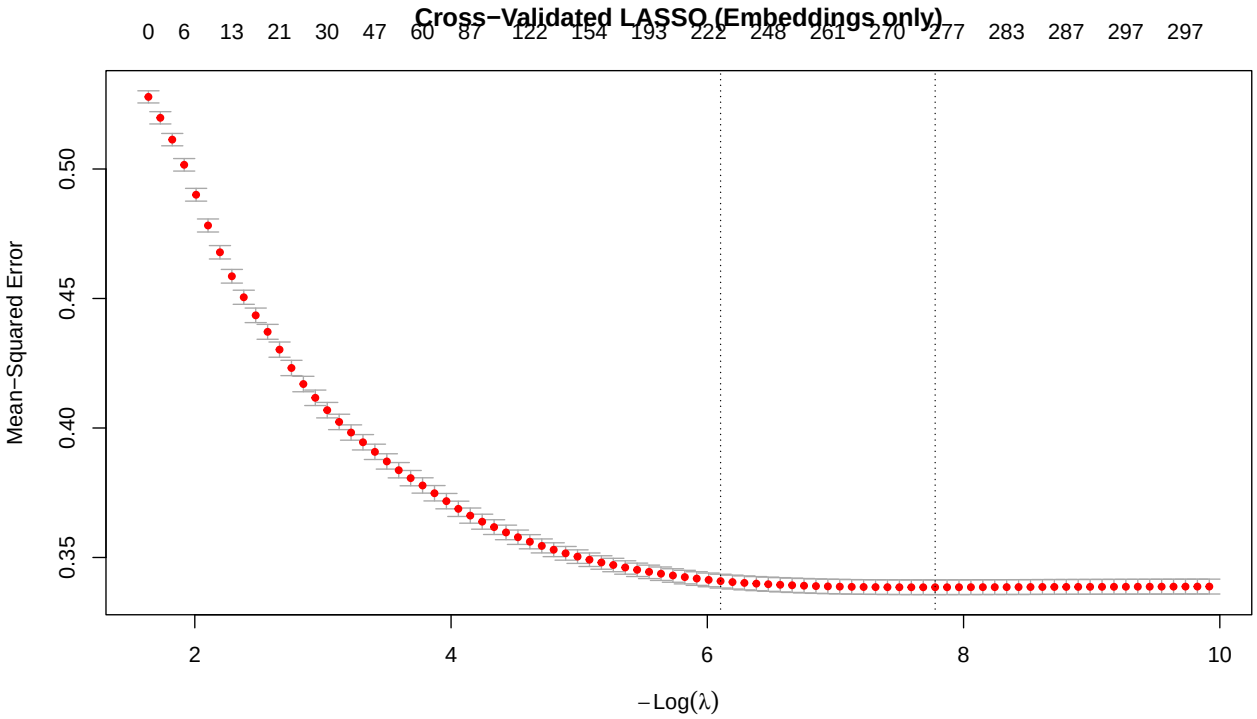
```
## [1] 11983   300
```

4.2. Fit LASSO Embeddings Model

```r
rmse = function(y, yhat) sqrt(mean((yhat - y)^2))
mae  = function(y, yhat) mean(abs(yhat - y))

train_Y = london_train$price_log
test_Y  = london_test$price_log

lasso_embed = cv.glmnet(
  x = vdat_train,
  y = train_Y
)

pred_log_embed = predict(lasso_embed, newx = vdat_test)[,1]

pred_price_embed = expm1(pred_log_embed)
actual_price = expm1(test_Y)

metrics_embed = tibble(
  model = "Embeddings only",
  rmse_log = rmse(test_Y, pred_log_embed),
  mae_log = mae(test_Y, pred_log_embed),
  rmse_dollar = rmse(actual_price, pred_price_embed),
  mae_dollar = mae(actual_price, pred_price_embed),
  n_features = ncol(vdat_train)
)

metrics_embed
```

```
## # A tibble: 1 x 6
##   model           rmse_log mae_log rmse_dollar mae_dollar n_features
##   <chr>              <dbl>   <dbl>       <dbl>      <dbl>      <int>
## 1 Embeddings only    0.593   0.461        155.       77.7        300
```

```r
plot(lasso_embed)
title("Cross-Validated LASSO (Embeddings only)")
```



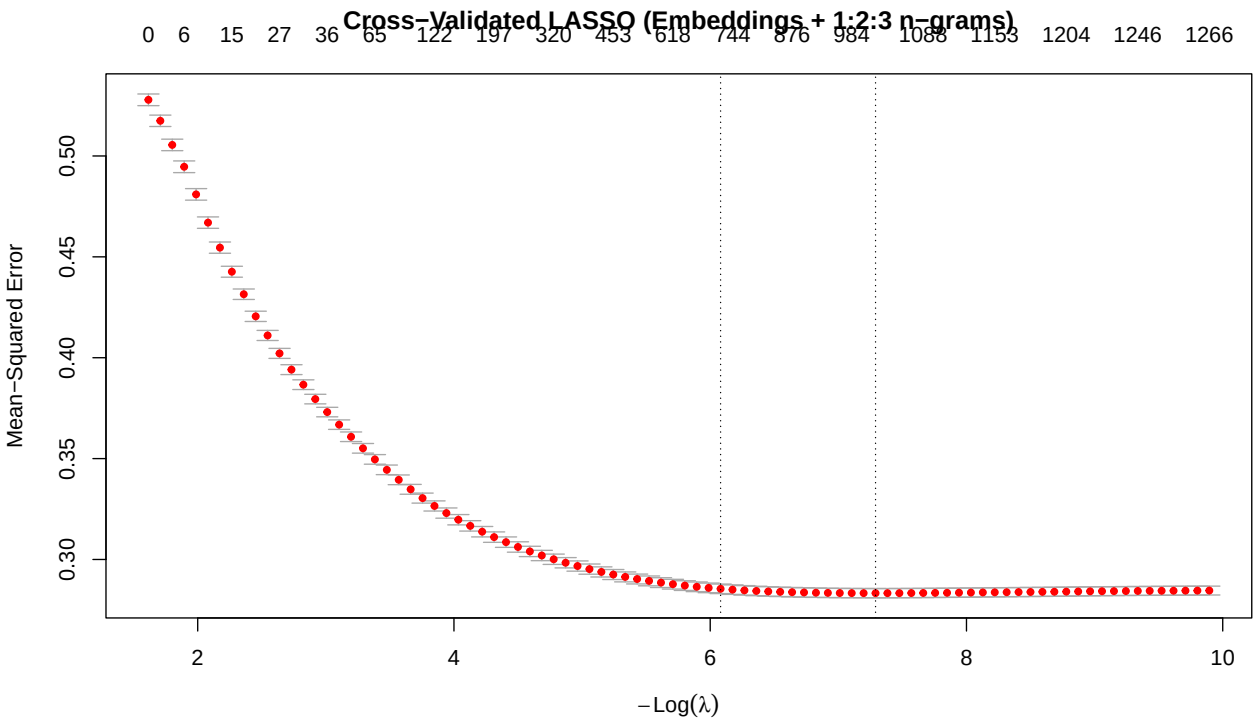
4.3. Fit LASSO Embeddings + 1:3 N-Grams

```
best_ngram = 1:3

dfm_train_ng = TMEF_dfm(london_train$description, ngrams = best_ngram)
dfm_test_ng  = TMEF_dfm(london_test$description,  ngrams = best_ngram, min.prop = 0) %>%
  dfm_match(colnames(dfm_train_ng))

X_train_ng = as.matrix(dfm_train_ng)
X_test_ng  = as.matrix(dfm_test_ng)

X_train_embed_ng = cbind(vdat_train, X_train_ng)
X_test_embed_ng  = cbind(vdat_test,  X_test_ng)

lasso_embed_ng = cv.glmnet(
  x = X_train_embed_ng,
  y = train_Y
)

pred_log_embed_ng = predict(lasso_embed_ng, newx = X_test_embed_ng)[,1]

pred_price_embed_ng = expm1(pred_log_embed_ng)

metrics_embed_ng = tibble(
  model = "Embeddings + 1:2:3 n-grams",
  rmse_log = rmse(test_Y, pred_log_embed_ng),
  mae_log = mae(test_Y, pred_log_embed_ng),
  rmse_dollar = rmse(actual_price, pred_price_embed_ng),
  mae_dollar = mae(actual_price, pred_price_embed_ng),
  n_features = ncol(X_train_embed_ng)
```

```
)

metrics_embed_ng
```

```
## # A tibble: 1 x 6
##   model                       rmse_log mae_log rmse_dollar mae_dollar n_features
##   <chr>                          <dbl>   <dbl>       <dbl>      <dbl>      <int>
## 1 Embeddings + 1:2:3 n-grams     0.543   0.417        121.       71.1       1312
```

```
plot(lasso_embed_ng)
title("Cross-Validated LASSO (Embeddings + 1:2:3 n-grams)")
```



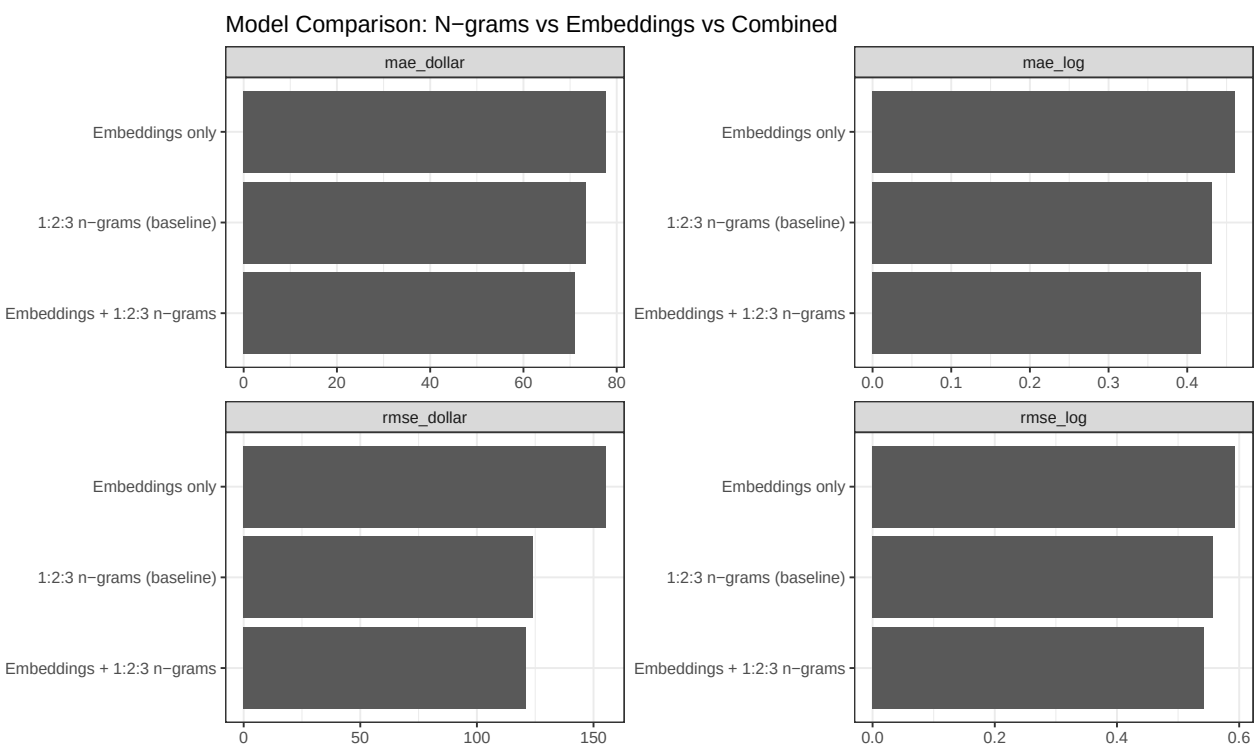
4.4. Compare Models (Embeddings, Embeddings+1:3 Grams, 1:3 Grams (baseline))

```
metrics_ng = results_desc %>%
  filter(ngram == "1:2:3") %>%
  transmute(
    model = "1:2:3 n-grams (baseline)",
    rmse_log = rmse_log,
    mae_log = mae_log,
    rmse_dollar = rmse_dollar,
    mae_dollar = mae_dollar,
    n_features = n_features
  )

compare_all = bind_rows(
  metrics_ng,
  metrics_embed %>% select(names(metrics_ng)),
  metrics_embed_ng %>% select(names(metrics_ng))
```

```
)

compare_all
```

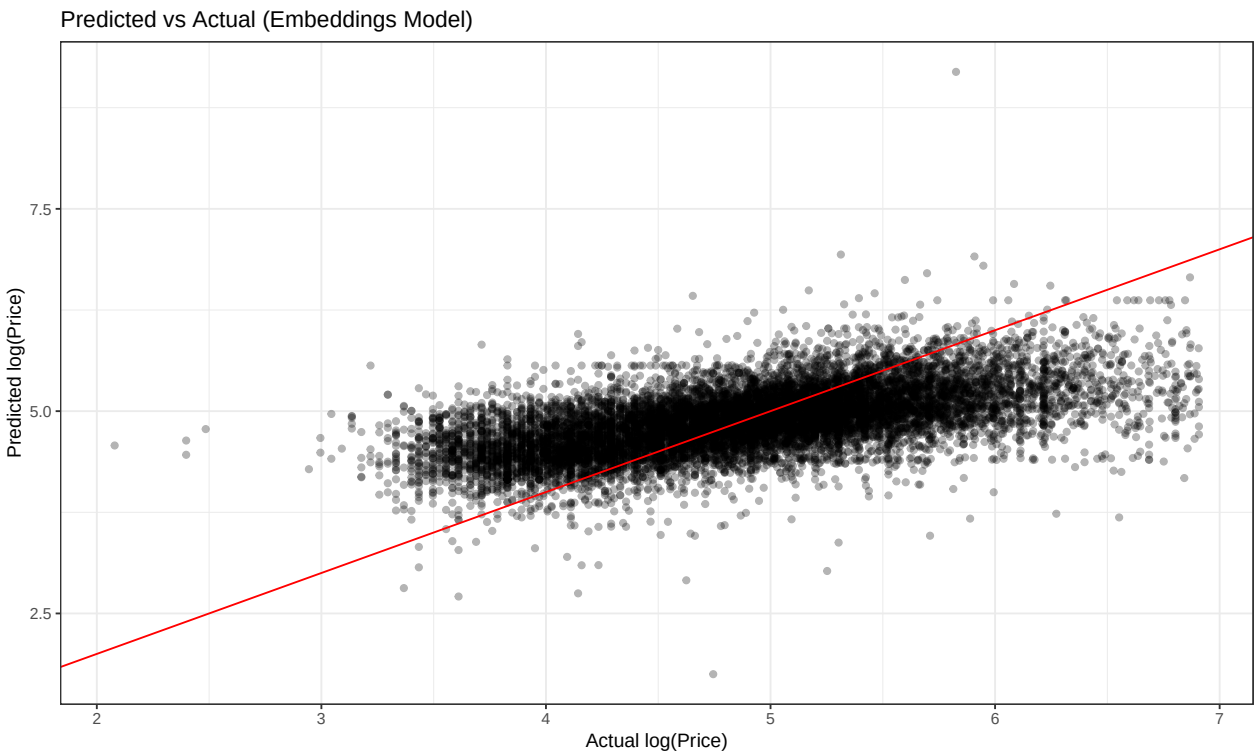
```
## # A tibble: 3 x 6
##   model                      rmse_log mae_log rmse_dollar mae_dollar n_features
##   <chr>                         <dbl>   <dbl>       <dbl>      <dbl>      <int>
## 1 1:2:3 n-grams (baseline)      0.557   0.431        124.       73.3       1012
## 2 Embeddings only               0.593   0.461        155.       77.7        300
## 3 Embeddings + 1:2:3 n-grams    0.543   0.417        121.       71.1       1312
```

```
compare_long = compare_all %>%
  pivot_longer(
    cols = c(rmse_log, mae_log, rmse_dollar, mae_dollar),
    names_to = "metric",
    values_to = "value"
  )

compare_long %>%
  ggplot(aes(x = reorder(model, value), y = value)) +
  geom_col() +
  coord_flip() +
  facet_wrap(~ metric, scales = "free") +
  theme_bw() +
  labs(title = "Model Comparison: N-grams vs Embeddings vs Combined",
       x = "",
       y = "")
```



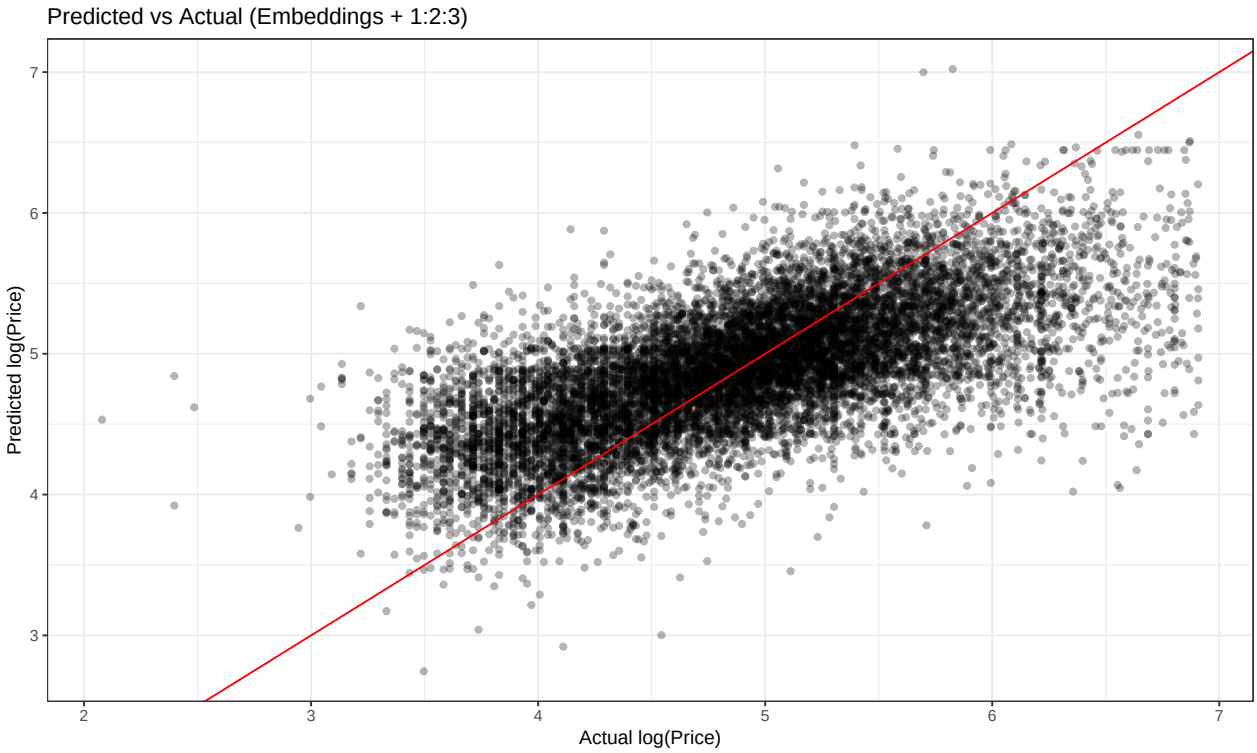

```
tibble(
  actual = london_test$price_log,
  predicted = pred_log_embed
```

```
) %>%
  ggplot(aes(x = actual, y = predicted)) +
  geom_point(alpha = 0.3) +
  geom_abline(slope = 1, intercept = 0, color = "red") +
  theme_bw() +
  labs(title = "Predicted vs Actual (Embeddings Model)",
       x = "Actual log(Price)",
       y = "Predicted log(Price)")
```

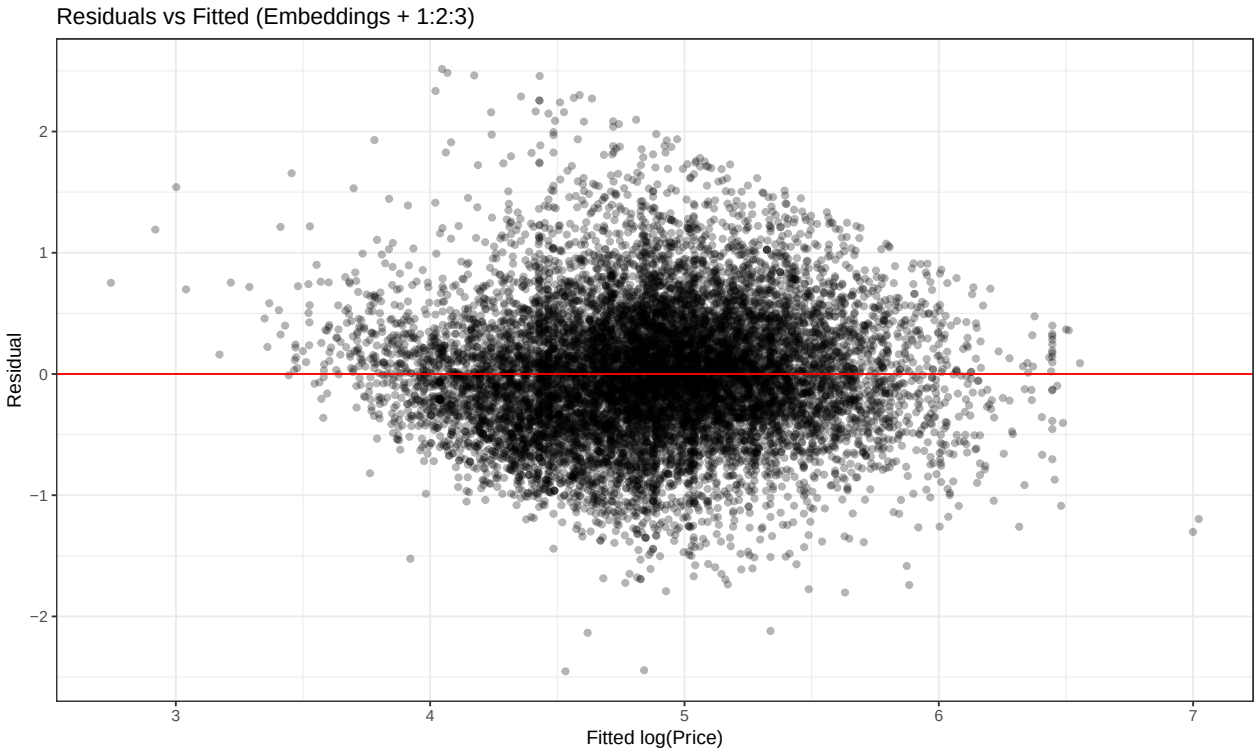


4.5. Diagnostics Plots for Best Model

```
best_pred_df = tibble(
  actual = london_test$price_log,
  predicted = pred_log_embed_ng
)

best_pred_df %>%
  ggplot(aes(x = actual, y = predicted)) +
  geom_point(alpha = 0.3) +
  geom_abline(slope = 1, intercept = 0, color = "red") +
  theme_bw() +
  labs(title = "Predicted vs Actual (Embeddings + 1:2:3)",
       x = "Actual log(Price)",
       y = "Predicted log(Price)")
```




```
best_resid_df = tibble(
  fitted = pred_log_embed_ng,
  residual = london_test$price_log - pred_log_embed_ng
)

best_resid_df %>%
  ggplot(aes(x = fitted, y = residual)) +
  geom_point(alpha = 0.3) +
  geom_hline(yintercept = 0, color = "red") +
  theme_bw() +
  labs(title = "Residuals vs Fitted (Embeddings + 1:2:3)",
       x = "Fitted log(Price)",
       y = "Residual")
```



SECTION 5 - Structured Features + Combined Model

Up to this point, we have evaluated text-only models. While descriptions contain meaningful economic signal, price determination in housing markets is likely driven primarily by structural characteristics such as location, size, and capacity. We therefore estimate a structured-only model and then a combined model incorporating both structured and textual features to assess the incremental value of text.

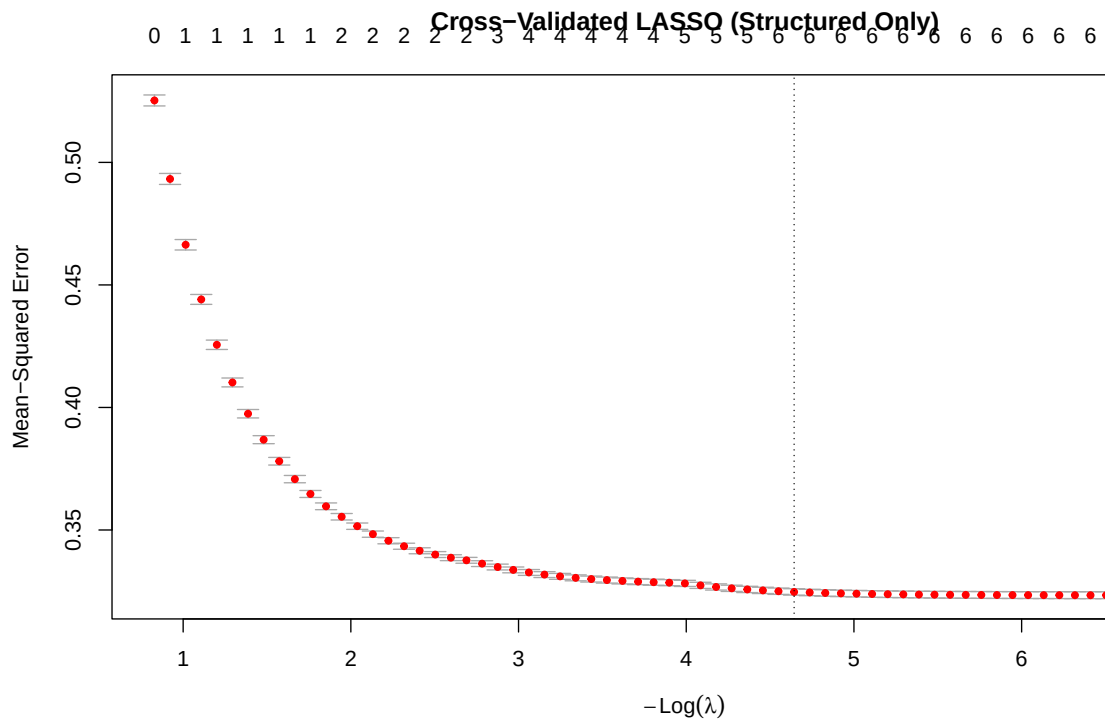
5.1. Structured-Only Model

```
struct_vars = c("latitude",
                "longitude",
                "accommodates",
                "bathrooms",
                "bedrooms",
                "beds")

struct_train = london_train %>%
  select(price_log, all_of(struct_vars)) %>%
  drop_na()

struct_test = london_test %>%
  select(price_log, all_of(struct_vars)) %>%
  drop_na()

train_struct = struct_train %>%
  select(all_of(struct_vars)) %>%
  as.matrix()
```

```r
test_struct = struct_test %>%
  select(all_of(struct_vars)) %>%
  as.matrix()

train_Y_struct = struct_train$price_log
test_Y_struct  = struct_test$price_log
```

5.1.1. Data Preparation

```r
lasso_struct = cv.glmnet(
  x = train_struct,
  y = train_Y_struct
)

plot(lasso_struct)
title("Cross-Validated LASSO (Structured Only)")
```



5.1.2. Model Estimation

```r
pred_log_struct = predict(lasso_struct, newx = test_struct)[,1]

pred_price_struct = expm1(pred_log_struct)
actual_price_struct = expm1(test_Y_struct)

metrics_struct = tibble(
  model = "Structured Only",
  rmse_log = sqrt(mean((test_Y_struct - pred_log_struct)^2)),
  mae_log = mean(abs(test_Y_struct - pred_log_struct)),
  rmse_dollar = sqrt(mean((actual_price_struct - pred_price_struct)^2)),
  mae_dollar = mean(abs(actual_price_struct - pred_price_struct))
```

```
)

metrics_struct
```

5.1.3. Test Performance

```
## # A tibble: 1 x 5
##   model           rmse_log mae_log rmse_dollar mae_dollar
##   <chr>              <dbl>   <dbl>       <dbl>      <dbl>
## 1 Structured Only    0.570   0.445        215.       78.4
```

5.2. Structured + Tex Model

```
london_train_id = london_train %>%
  mutate(row_id = row_number())

london_test_id = london_test %>%
  mutate(row_id = row_number())
```

```
struct_vars = c("latitude","longitude","accommodates","bathrooms","bedrooms","beds")

struct_train = london_train_id %>%
  select(row_id, price_log, all_of(struct_vars)) %>%
  drop_na()

struct_test = london_test_id %>%
  select(row_id, price_log, all_of(struct_vars)) %>%
  drop_na()
```

```
keep_train_ids = struct_train$row_id
keep_test_ids = struct_test$row_id
```

```
train_struct = struct_train %>%
  select(all_of(struct_vars)) %>%
  as.matrix()

test_struct = struct_test %>%
  select(all_of(struct_vars)) %>%
  as.matrix()

train_Y_struct = struct_train$price_log
test_Y_struct = struct_test$price_log
```

```
vdat_train_aligned = vdat_train[keep_train_ids, ]
vdat_test_aligned  = vdat_test[keep_test_ids, ]

X_train_ng_aligned = X_train_ng[keep_train_ids, ]
X_test_ng_aligned  = X_test_ng[keep_test_ids, ]
```

5.2.0.

```
X_train_combined = cbind(train_struct, vdat_train_aligned, X_train_ng_aligned)
X_test_combined  = cbind(test_struct,  vdat_test_aligned,  X_test_ng_aligned)

dim(X_train_combined)
```

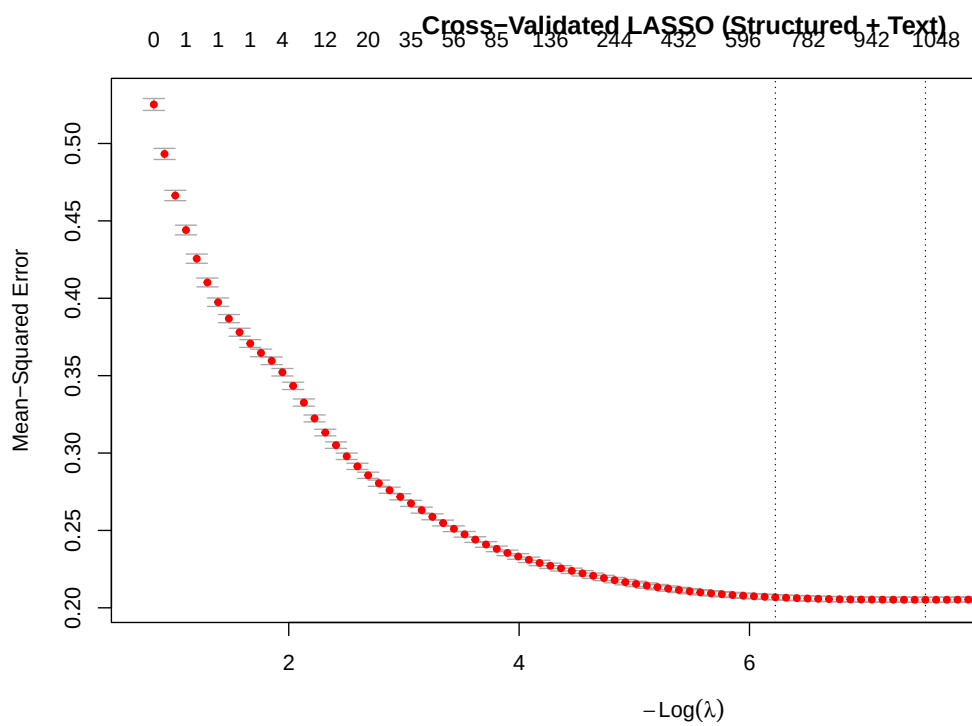
5.2.1. Combine Feature

```
## [1] 47700  1318
```

```
dim(X_test_combined)
```

```
## [1] 11926  1318
```

```
lasso_combined = cv.glmnet(
  x = X_train_combined,
  y = train_Y_struct
)

plot(lasso_combined)
title("Cross-Validated LASSO (Structured + Text)")
```



5.2.2. Estimated Combined Model

```
pred_log_combined = predict(lasso_combined, newx = X_test_combined)[,1]

pred_price_combined = expm1(pred_log_combined)

metrics_combined = tibble(
  model = "Structured + Text",
  rmse_log = sqrt(mean((test_Y_struct - pred_log_combined)^2)),
  mae_log = mean(abs(test_Y_struct - pred_log_combined)),
```

```r
  rmse_dollar = sqrt(mean((actual_price_struct - pred_price_combined)^2)),
  mae_dollar = mean(abs(actual_price_struct - pred_price_combined))
)

metrics_combined
```

5.2.3. Evaluation

```
## # A tibble: 1 x 5
##   model             rmse_log mae_log rmse_dollar mae_dollar
##   <chr>                <dbl>   <dbl>       <dbl>      <dbl>
## 1 Structured + Text    0.464   0.350        844.       69.4
```

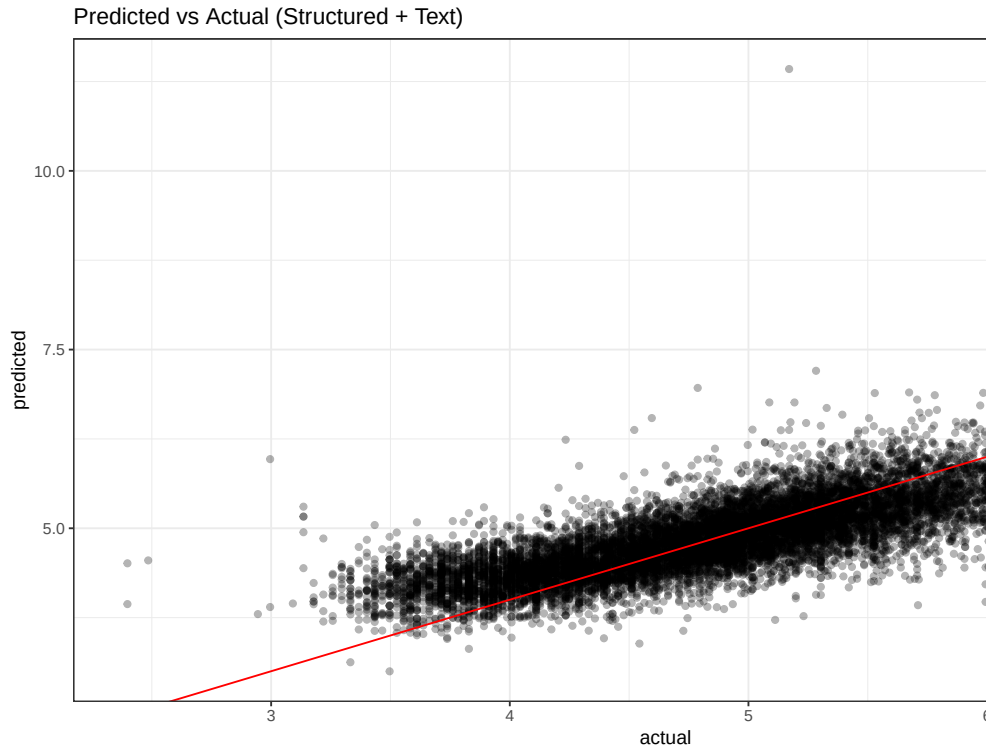
5.3. Model Comparison

```r
compare_section5 = bind_rows(
  metrics_struct,
  metrics_embed_ng,
  metrics_combined
)

compare_section5
```

```
## # A tibble: 3 x 6
##   model                      rmse_log mae_log rmse_dollar mae_dollar n_features
##   <chr>                         <dbl>   <dbl>       <dbl>      <dbl>      <int>
## 1 Structured Only               0.570   0.445        215.       78.4         NA
## 2 Embeddings + 1:2:3 n-grams    0.543   0.417        121.       71.1       1312
## 3 Structured + Text             0.464   0.350        844.       69.4         NA
```

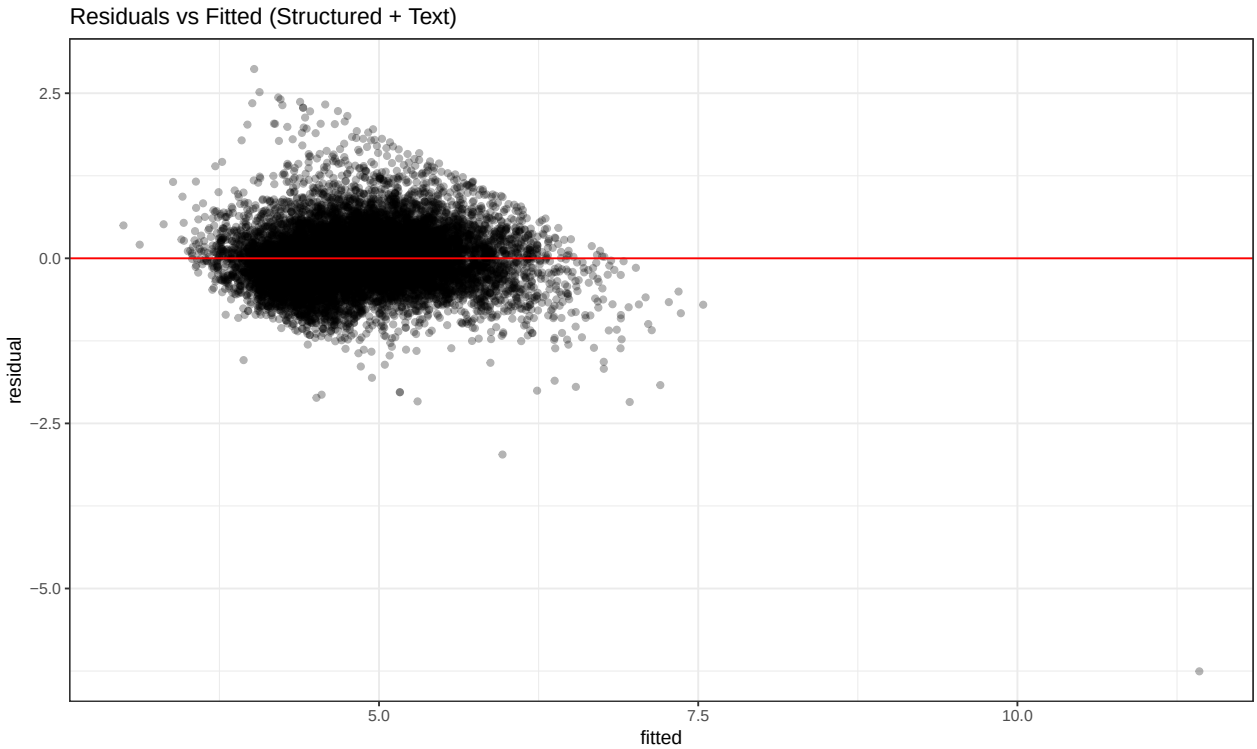
5.4.

```r
tibble(
  actual = test_Y_struct,
  predicted = pred_log_combined
) %>%
  ggplot(aes(x = actual, y = predicted)) +
  geom_point(alpha = 0.3) +
  geom_abline(slope = 1, intercept = 0, color = "red") +
  theme_bw() +
  labs(title = "Predicted vs Actual (Structured + Text)")
```



5.4.1. Diagnostics for Best Model

```
tibble(
  fitted = pred_log_combined,
  residual = test_Y_struct - pred_log_combined
) %>%
  ggplot(aes(x = fitted, y = residual)) +
  geom_point(alpha = 0.3) +
  geom_hline(yintercept = 0, color = "red") +
  theme_bw() +
  labs(title = "Residuals vs Fitted (Structured + Text)")
```



The residual plot for the combined structured and text model shows no strong nonlinear pattern and residuals are centered around zero. Variance appears relatively stable across fitted values, with only mild dispersion at higher predicted prices. Apart from a small number of extreme observations, residual behavior is largely random, indicating that the model is well-specified given the available features.